

WebSocket 实现

WebSocket 实现

概述

所有 WebSocket 通信使用 OGS 公开 WebSocket 服务端：

wss://wsp.online-go.com

整个项目没有 Rust 后端 WebSocket 代码，全部在 TypeScript 前端实现，依赖浏览器原生 WebSocket API。

原则：只读连接公开棋局/Review，不做登录 OAuth、不发评论、不回写 OGS、不调用 OGS AI。

架构

两个 WebSocket 消费者

类	文件	生命周期	用途
OGSConnector	src/ogs/OGSConnector.ts	持久 (App mount -> unmount)	同步指定 game/ review 的棋步
OGSBrowserService	src/ogs/ OGSBrowserService.ts	临时 (每次请求新 建)	查询公开对局列表

数据流

App.tsx (useEffect mount)

-> new OGSConnector()

-> onConnectionStatusChanged -> setOgsStatus / setOgsStatusDetail

-> onMovesUpdated -> applyOgsMoveUpdate (更新棋盘)

OGSConnector 在 App.tsx:374 创建，通过 useRef 持有，组件卸载时调用 disconnect()。

OGSConnector 详解 (核心)

连接生命周期

connectGame(gameId) / connectDemo(demoId)

-> disconnect() 清理旧连接

- > openSocket() 创建 WebSocket
- > onopen: 发送 review/connect 或 game/connect
- > onmessage: handleMessage 路由到各 handler
- > onerror: 日志 + emitStatus("error")
- > onclose: 自动重连 (最多3次, 间隔1.5s)

协议消息

发送 (src/ogs/OGSConnector.ts):

场景	消息
连接 Review	["review/connect", { review_id: <id> }] + ["chat/join", { channel: "review-<id>" }]
连接 Game	["game/connect", { chat: false, game_id: <id> }]
断开 Review	["review/disconnect", { review_id: <id> }]
断开 Game	["game/disconnect", { game_id: <id> }]

接收 (src/ogs/OGSConnector.ts:209-267):

事件	处理
game/<id>/gamedata	processGameData -> 解码 moves -> 回调
game/<id>/move	触发 REST API 重新拉取完整数据
game/<id>/error	日志 + emitStatus("error")
review/<id>/r	processDemoMessage -> 解码棋步 -> append/replace mainline
review/<id>/full_state	全量状态同步
review/<id>/error	日志 + emitStatus("error")

重连机制

```
const MAX_RECONNECT_ATTEMPTS = 3;
const RECONNECT_DELAY_MS = 1500;
```

- 收到 close 事件时检查 shouldReconnect 标记
- 重连计数 reconnectAttempts 每次 open 时归零
- 超过 3 次后设置 shouldReconnect = false, 不再重连

Demo 降级机制

```
const DEMO_STATIC_FALLBACK_DELAY_MS = 5000;
```

- connectDemo 时启动 5s 计时器
- 如果 5s 内未收到直播数据 (demoUsedLiveSocket 为 false 且 demoMoves 为空), 调用 loadStaticDemo 通过 REST API 获取 SGF:

GET <https://online-go.com/api/v1/reviews/<demoId>/sgf?without-comments=1>

- 收到直播数据后立即取消计时器

Game 轮询机制

```
const GAME_POLL_INTERVAL_MS = 10_000;
```

- 每 10s 调用 REST API 作为补充:

```
GET https://online-go.com/api/v1/games/<gameId>
```

- 棋局结束后停止轮询 (phase === "finished" 或 outcome 非空)

棋步去重

```
function makeMoveSignature(moves: ReviewMove[]): string {
  const lastMove = moves[moves.length - 1];
  return lastMove
    ? `${moves.length}:${lastMove.color}:${lastMove.x}:${lastMove.y}`
    : "0";
}
```

processGameData 中比较 lastGameMoveSignature, 相同则不触发更新。

OGSBrowserService 详解

用于浏览公开对局列表 (OgsBrowserDialog)。

临时 WebSocket 模式

```
function sendOgsSocketRequest<T>(
  eventName: string,
  payload: unknown
): Promise<T> {
  return new Promise((resolve, reject) => {
    const socket = new WebSocket(OGS_WEBSOCKET_URL);
    // 8s 超时
    // open -> authenticate -> send gamelist/query
    // message -> 解析响应 ID -> resolve
    // error -> reject
  });
}
```

每次查询新建一个连接, 查询完立即关闭。超时 8s。

鉴权

```
async function authenticateOgsSocket(socket: WebSocket): Promise<void> {
  const response = await fetch("https://online-go.com/api/v1/ui/config");
```

```

const config = await response.json(); // { user_jwt?: string }
socket.send(
  JSON.stringify([
    "authenticate",
    {
      client: "TensuGo",
      jwt: config.user_jwt ?? "",
      language: "en",
      user_agent: window.navigator.userAgent,
    },
  ])
);
}

```

从 REST API 获取匿名 JWT，发送 authenticate 消息。无需用户登录。

查询参数

```

queryGameList("live" | "corr", {
  channel: "tensugo-watch",
  from: 0,
  limit: 100,
  list,
  sort_by: "rank",
  where: {
    hide_13x13,
    hide_19x19,
    hide_9x9,
    hide_other,
    hide_bot_games: true,
  },
});

```

棋步解码

OGS 编码格式

OGS 使用字母编码坐标：

```
const OGS_COORDINATE_SEQUENCE = "abcdefghijklmnopqrstuvwxyz";
```

- 每 2 个字符 = 1 手棋 (x, y)
- . 表示 pass/无效坐标 (ogsCharToNumber 返回 -1)
- 可选 ! 前缀表示编辑模式，如 !1 表示黑子编辑

解码流程

ogsMoveString (如 "qdddpqdqndpoqoqnp...")
-> decodeRawOgsMoves (解析成 {color, x, y}[])
-> 每 2 字符一对, 遇到 "!" 处理编辑标记
-> decodeOgsMoveString (转为 ReviewMove[])
-> 过滤越界坐标
-> 补充 moveNumber
-> 生成 warnings

Game 开局数据解码

decodeOgsGameMoves (OGSConnector.ts:577) 处理 REST API 返回的 moves 数组:

```
// API 格式: [[x, y, timestamp, color?], ...]  
const [x, y] = rawMove;  
const colorMarker = rawMove[3]; // 0=black(默认), 1=black, 2=white
```

类型系统

类型	文件	用途
OgsConnectionStatus	types.ts:7	"idle" \ "connecting" \ "connected" \ "syncing" \ "disconnected" \ "unsupported" \ "error"
OgsMoveUpdate	types.ts:23	棋步更新事件负载, 含 moves/metadata/demold/gameId
OgsStatusUpdate	types.ts:39	连接状态事件负载
OgsUrlTarget	types.ts:3	URL 解析结果, {kind:"demo", demoId} 或 {kind:"game", gameId}
OgsDecodedMoves	types.ts:16	解码后的棋步数据

URL 解析

```
// src/ogs/ogsUrl.ts  
parseOgsUrl("https://online-go.com/review/1730972");  
// -> { kind: "demo", demoId: 1730972 }
```

```
parseOgsUrl("https://online-go.com/game/123456");  
// -> { kind: "game", gameId: 123456 }
```

```
parseOgsUrl("review:1730972");  
// -> { kind: "demo", demoId: 1730972 }
```

支持完整 URL 和简写格式 (review:<id>, game:<id>, demo:<id>)。

API 端点汇总

用途	方法	URL
WebSocket	WS	wss://wsp.online-go.com
获取匿名 JWT	GET	https://online-go.com/api/v1/ui/config
Game 详情	GET	https://online-go.com/api/v1/games/{id}
Review SGF	GET	https://online-go.com/api/v1/reviews/{id}/sgf

文件索引

文件	职责
src/ogs/OGSConnector.ts	核心 WebSocket 连接器, 持久连接, 同步 game/review 棋步
src/ogs/OGSBrowserService.ts	临时 WebSocket 查询公开对局列表
src/ogs/types.ts	WebSocket 相关类型定义
src/ogs/ogsMoveDecoder.ts	OGS 编码棋步解码器
src/ogs/ogsUrl.ts	OGS URL 解析
src/ogs/ogsRank.ts	OGS 段位格式化/过滤工具
src/app/App.tsx	创建/持有 OGSConnector, 连接状态和棋步更新的消费者
src/components/OgsDialog.tsx	“Open OGS URL”弹窗
src/components/OgsBrowserDialog.tsx	“OGS Browser”弹窗, 浏览公开对局
Prompt/OGS 直播.txt	原始设计文档

注意事项

1. 无后端 WebSocket: 所有 WebSocket 代码在 TypeScript 前端, Rust 后端不参与。
 2. 单连接: OGSConnector 同时只维护一个 WebSocket, 新连接前会 disconnect() 旧的。
 3. 低频原则: game 轮询间隔 10s, review 不主动轮询 (靠 Server Push)。
 4. 匿名访问: 不需要用户登录 OGS, 通过匿名 JWT 鉴权。
 5. 降级优先: Review 优先走 WebSocket 直播, 5s 无数据则降级为 REST API 拉取 SGF。
 6. 安全性: 所有通信走 WSS/HTTPS, 不存凭据, 不写 OGS。
-

Pandoc 转换

将本文件转为 PDF:

```
pandoc docs/tech-detail/websocket.md -o websocket.pdf \  
--pdf-engine=xelatex \  
-V CJKmainfont="PingFang SC" \  
-V mainfont="PingFang SC" \  
-V monofont="Menlo"
```

如果上述字体不可用，替换为系统上已安装的任意中文字体（如 "STHeiti"、"Noto Sans CJK SC"）。